USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

Precise and Effective Gadget Chain Mining through Deserialization Guided Call Graph Construction

Yiheng Zhang, Ming Wen, and Shunjie Liu, *Huazhong University of Science and Technology;* Dongjie He, *Chongqing University;* Hai Jin, *Huazhong University of Science and Technology*

https://www.usenix.org/conference/usenixsecurity25/presentation/zhang-yiheng

This paper is included in the Proceedings of the 34th USENIX Security Symposium.

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the 34th USENIX Security Symposium is sponsored by USENIX.

Precise and Effective Gadget Chain Mining through Deserialization Guided Call Graph Construction

Yiheng Zhang[1,2], Ming Wen[1,2,*], Shunjie Liu[2], Dongjie He[4], and Hai Jin[1,3]

[1]*National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Huazhong University of Science and Technology (HUST)*
[2]*Hubei Engineering Research Center on Big Data Security, Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, HUST, China*
[3]*Cluster and Grid Computing Lab, School of Computer Science and Technology, HUST, China*
[4]*Chongqing University, China*

Abstract

Gadget chains of consecutive method invocations can trigger Java deserialization vulnerabilities, posing significant security threats to applications. Existing detection methods often rely on imprecise call graphs while overlooking complex features like deserialization, dynamic proxy, and reflection, leading to incomplete and inaccurate results.

This paper presents FLASH, a novel gadget chain detection tool leveraging a deserialization-guided call graph construction approach. FLASH begins with controllability analysis to determine if variables can be restored through deserialization. It then applies a hybrid dispatch technique to resolve callees at call sites based on the controllability of their receiver variable: if controllable, a more comprehensive but potentially less precise technique (e.g., *CHA*, *proxy dispatch*) is used; otherwise, a more precise technique (e.g., *pointer analysis*) is applied. FLASH also recovers missing call edges by handling reflection involving controllable variables, and prunes false or irrelevant edges to improve accuracy and efficiency.

Evaluation on 30 applications demonstrates that FLASH achieves higher recall (with 30.8% lower false negative rate) and precision (with 25.9% lower false positive rate) than state-of-the-art methods, at the cost of limited overhead. Furthermore, FLASH detects a total of 90 new gadget chains and 10 existing CVEs. Leveraging the new gadget chains, FLASH uncovers 5 previously unknown exploitation methods of the vulnerabilities, which demonstrate a broader impact compared to previously known CVEs.

1 Introduction

Java Deserialization Vulnerability (JDV) [12] arises when an attacker crafts input data that, upon deserialization, triggers a sequence of class methods to eventually reach security-sensitive methods. JDV is a common and well-known vulnerability type, accounting for 30% of the detected vulnerabilities among all Java applications [52]. It can lead to various attacks, including remote code execution (RCE), file deletion (FD), file write (FW), and server-side request forgery (SSRF). In fact, the prevalence of deserialization vulnerabilities is also highlighted by the 2024 CWE Top 10 list [31], which includes 'Deserialization of Untrusted Data' as a critical issue. Therefore, the detection and mitigation of JDVs remain a relevant and important topic in application security.

Gadget chains, method sequences that lead to security-sensitive methods (i.e., sinks) starting from deserialization-related methods (i.e., sources), are the key to detecting and mitigating JDVs. By preventing the execution of gadget chains through source code patching [42], JDVs can be restricted. For example, JDK patched the `AnnotationInvocationHandler` class by assigning a new object to a variable, successfully preventing attackers from exploiting the variable to further trigger a JDV [32]. Therefore, detecting potential gadget chains in Java applications is crucial, which is the main focus of this study.

Existing approaches. Existing static detection tools [8, 12, 17, 28, 30, 53] and fuzzing-based dynamic verification tools [9–11, 21, 28, 35, 48] detect gadget chains by identifying source-to-sink paths on call graphs, typically constructed through either precise but resource-intensive analyses (e.g., *Pointer Analysis* (PA), which determines the objects that a variable may point to) [35] or less precise but lighter analyses (e.g., *Class Hierarchy Analysis* (CHA), which utilizes the declared type of a variable). The former often overlooks runtime polymorphism introduced by deserialized objects [10, 35], leading to unsound call graphs which lacks call paths that actually occur at runtime (i.e., *missing edges*). Furthermore, the *whole-program* pointer analysis [4, 5] style that analyzes the points-to relationships of all variables in the program suffers from heavy overhead, especially for large applications. On the other hand, the latter introduces numerous edges that are unreachable during deserialization (i.e., *fake edges*), can hinder efficient searching and may even result in false positives. Additionally, current methods rarely consider reflection and dynamic proxy in deserialization, failing to analyze reflection targets or whether a deserialized variable pointing to a proxy

*Ming Wen is the corresponding author

object is capable of invoking handler methods (see Section 2.3 for details), which also results in missing edges. As a result, existing analyses frequently lead to both false positives and negatives in gadget chain detection.

Challenges. We identify three key challenges in detecting gadget chains: ***C1:*** *Deserialization-introduced runtime polymorphism.* The deserialization process is more complex than traditional static analysis [18, 23, 24, 50], as it involves objects instantiated via both `new` statements and fields implicitly restored during deserialization. When resolving virtual calls, pointer analysis alone cannot capture deserialized objects, while CHA results in many false positives. Balancing pointer analysis's precision with CHA's soundness is critical for accurate deserialization call graphs. ***C2:*** *Advanced dynamic features.* Java reflection is inherently difficult to handle [26, 44] and rarely supported by existing researches [9, 10, 12, 17]. Additionally, identifying which variables can be dynamically proxied during deserialization is challenging, as unsupported variables will cause exceptions, disrupting gadget chain execution (see Section 2.3). To our best knowledge, no research has yet analyzed dynamic proxy during deserialization. ***C3:*** *Efficiency.* Processing these features without introducing significant overhead presents another challenge to be solved.

Solution. We present FLASH, a new gadget chain detection tool that leverages a novel deserialization-guided call graph construction approach. To address **C1**, we propose a hybrid dispatch technique to resolve callees at call sites based on the controllability of the receiver variable, which is determined using a deserialization-driven controllability analysis based on Context-Free Language (CFL) reachability. Specifically, if the receiver variable is controllable by attackers during deserialization (i.e., tainted variable), FLASH uses CHA (comprehensive but less precise) to resolve callees by considering that attackers can potentially instantiate the receiver variable with any subclass via deserialization. Otherwise, FLASH leverages pointer analysis (more precise) to resolve callees based on the objects the receiver may reference. To address **C2**, FLASH models dynamic proxy behavior as a method dispatch process using static analysis. It identifies call sites that may trigger proxy behaviors based on a set of dedicated features and incorporates proxy jump edges into the call graph. For reflection handling, FLASH adopts a controllability-based approach, analyzing controllable strings and JavaBean properties to identify potential reflection targets and adding reflection jump edges to enhance call graph soundness and gadget chain detection accuracy. To address **C3**, FLASH implements controllability analysis in a demand-driven manner, querying only variables of interest. Consequently, FLASH detects gadget chains with high accuracy and limited overhead.

Evaluation. We evaluated FLASH on a well-known benchmark [15, 29] combined with our newly collected dataset. The new dataset was collected by reviewing pull requests of the original dataset and searching for JDV-related exploits in top-ranked projects of the Maven repository. Results show that FLASH successfully identifies a total of 90 previously undiscovered gadget chains as well as 10 known CVEs. By analyzing the newly discovered gadget chains, FLASH uncovers 5 new exploitation methods of the vulnerabilities, which can bypass existing patches and pose greater risks compared to previously known CVEs. More importantly, FLASH reduces the false negative rate by 30.8% and the false positive rate by 25.9% (excluding JDD) compared to the state-of-the-art [11, 12, 48]. Furthermore, 24.5% of call sites are resolved with higher precision using our hybrid dispatch strategy and 4.8% of call sites may trigger dynamic proxy, highlighting the impact of our key designs. These results demonstrate FLASH's superior performance in detecting gadget chains. All newly discovered gadget chains have been reported to the relevant projects and open-sourced to support future research.

This paper makes the following contributions:

- ***Controllability Analysis.*** We propose a novel CFL-reachability-based Deserialization Driven Controllability Analysis (DDCA) to efficiently determine the controllability of variables on-demand, specifically for Java deserialization scenarios.
- ***More Complete and Precise Deserialization Call Graph.*** The hybrid dispatch technique, combined with reflection processing, enables FLASH to generate a more complete and precise call graph for gadget chain detection.
- ***Comprehensive Results.*** FLASH outperforms the state-of-the-art, reducing false negatives by 30.8% and false positives by 24.3%. It also uncovers a total of 90 new gadget chains and 10 existing CVEs.
- ***Implementation.*** We present our idea as FLASH, with the artifact available at https://doi.org/10.5281/zenodo.15606159. Additional materials, including experimental results are available on https://github.com/CGCL-codes/Flash to facilitate future research.

2 Background and Motivation

2.1 Java Deserialization Vulnerability

Serialization and Deserialization: Serialization involves converting an in-memory object to a byte stream or text form by encoding its fields using a specific algorithm. Deserialization reverses this process, reconstructing the object by parsing its fields and performing necessary restoration operations. Many programming languages, including Java, offer built-in mechanisms or libraries to facilitate these processes. In Java, deserialization is frequently employed in remote procedure calls (RPC) and data transfer between web applications [12]. However, deserialization is inherently open and dynamic [9], with its behavior varying based on the input data. Without proper security measures, attackers can exploit this process

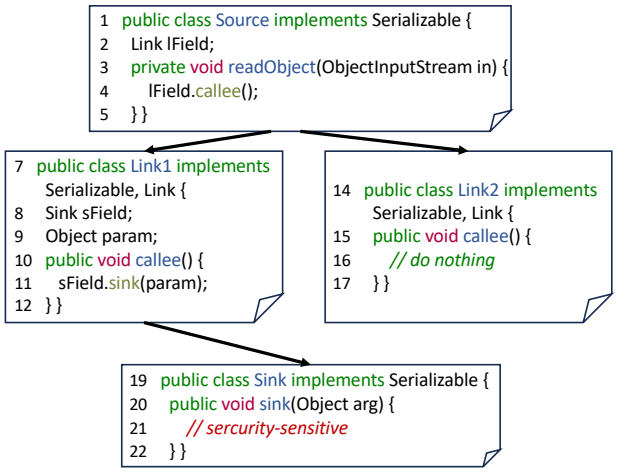

Figure 1: An example of a simplified gadget chain

to hijack the execution flow, invoke dangerous methods, and ultimately trigger *Java deserialization vulnerabilities*.

Controllable Variables: When an application deserializes the data stream from an attacker, the recovered objects and the fields of a class are generally controllable by the attacker [12]. For example, the results of operations on the stream (such as `readString` and so on) are generally controllable.

Gadget and Gadget Chain: A *gadget* is a method that uses controllable variables [42]. We use the example in Figure 1 to explain the concept of gadget and gadget chain. The default deserialization mechanism in JDK will invoke a class's overridden `readObject` method, which is generally the source gadget (line 3 in Figure 1). Then at the call site on line 4, there exist two candidate callees: `Link1#callee` and `Link2#callee`. Since `Link2#callee` does not use any controllable variables, it is not considered as a gadget. While `Link1#callee` uses controllable fields *sField* and *param*. Therefore, an attacker can choose to invoke `Link1#callee` by controlling the field `lField` of class `Source` as an object of class `Link1`. An attacker can repeat this process until reaching a security-sensitive method in the application (`Sink#sink` in Figure 1). The chain of consecutive calls formed by such gadgets is referred to as a ***gadget chain***. For instance, in the example in Figure 1, `Source#readObject` → `Link1#callee` → `Sink#sink` is a gadget chain whereas `Source#readObject` → `Link2#callee` is not.

2.2 Demand-Driven Points-To Analysis

Pointer analysis maps each variable to the abstract objects it may reference at runtime by computing the points-to relations, which holds significant importance for analysis and optimization in object-oriented languages such as Java [47]. Based on the analysis scope, pointer analysis can be categorized into *whole-program pointer analysis* [4,5] and *demand-driven pointer analysis* [19,43,47,49,54]. Whole-program pointer analysis conducts exhaustive analysis of programs, while demand-driven pointer analysis focuses on the computations of certain interested variables, thereby being more efficient and less resource-consuming.

A well-known demand-driven pointer analysis approach is built upon solving the Pointer Assignment Graph (PAG) [54] reachability problem [47]. In this scenario, determining the result of a variable's points-to information is equivalent to identifying which objects can flow to that variable in the PAG. However, certain factors will affect the precision of the analysis. For instance, in Java, field-sensitivity [54] requires that once an object o is written into the field of a variable x, it can only be accessed through a field read operation on variables that are alias (point to the same object) with x. Therefore, existing research leverage Context-Free Language (CFL) reachability to match field `read` and `write` operations [43,46,47].

CFL-Reachability is an extension of traditional graph reachability [38], primarily used to address the balanced parenthesis problems (e.g., ensuring context sensitivity in inter-procedural analysis) [47]. In the CFL-Reachability problem, the edge labels of a directed graph are derived from the terminal symbols of a context-free grammar. If a path can be generated by this grammar, then the path is considered reachable in the corresponding context-free language.

2.3 Reflection and Dynamic Proxy Usage

The usages of reflection and dynamic proxy complicate the deserialization process, thus presenting significant challenges for precise call graph construction.

Java Reflection APIs are frequently used in practice, allowing an application to decide which functionality to invoke at runtime [26]. During deserialization, if the arguments of the reflection call site are controllable, an attacker can use reflection to invoke intended methods, thereby altering the control flow to malicious methods. For example, A certain number of gadget chains exploit reflection to invoke the '`getter`' methods (i.e., methods whose name begins with '`get`', see Section 2.4 for more details) to reach sinks.

On the other hand, the dynamic proxy API allows users to define *invocation handler* code and proxy classes, primarily used to support dynamic loading [14]. In this paper, methods formed by such code are referred to as *handler methods*, and method calls on the proxy object will be forwarded to the corresponding handler method. Dynamic proxy classes that implement the *java.io.Serializable* interface may complicate the deserialization process. For example, at line 4 of Figure 1, an attacker can proxy the `lField` to be an instance of a proxy class for interface `Link`, potentially invoking the proxy class's handler method during deserialization. However, not all controllable call points can trigger dynamic proxy behaviors. Attackers can only proxy receiver variables for controllable call points with specific features. Otherwise, the payload genera-

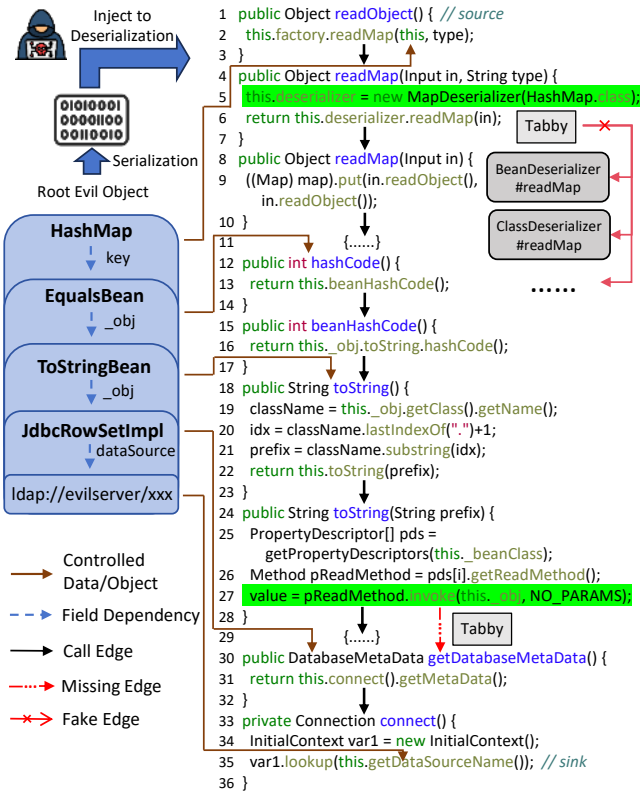

Figure 2: A simplified motivating example of the Hessian-Rome gadget chain

tion or the deserialization process will be terminated, leading to unsuccessful exploitation. For example, consider the call site at line 11 in Figure 1 (an *invoke virtual* call). Proxying `sField` of class `Link1` will throw a `ClassCastException` during payload generation. Based on this observation, we manually collect a set of features for call sites that can trigger dynamic proxy behaviors (see Section 3.3).

2.4 A Motivating Example

Over recent years, plenty of effective approaches on mining gadget chains have been proposed [8–12, 17, 21, 28, 30, 35, 44, 48], which typically identify gadget chains based on call graphs. However, existing methods suffer from two critical limitations when constructing call graphs: *fake edges* and *missing edges*. Fake edges represent unreachable call paths, while missing edges fail to capture necessary control flow paths. Both issues lead to imprecise call graphs, resulting in false positives and negatives in gadget chain detection.

Figure 2 shows a real-world gadget chain from the Hessian [2] and Rome [1] projects. To exploit this JDV, the attacker can construct a malicious object starting from the sink in a backward manner. As shown in Figure 2, the attacker can set the `dataSource` field of a `JdbcRowSetImpl` object to be an evil address, and then assigns this object to the `_obj` field of a `ToStringBean` object. By repeating this process, the attacker ultimately obtains a nested `HashMap` object, which can be serialized to produce a malicious payload. During deserializing the payload, the process begins from line 1 and invokes the `readMap` method at line 4, which instantiates a `MapDeserializer` object and again calls `readMap` at line 8. At line 9, a recovered object (i.e., the `EqualsBean` object) is added into a map, which subsequently (line 11 omits this process) triggers its `hashCode` method. Since the `_obj` field of the `EqualsBean` object points to the `ToStringBean` object, the `toString` method at line 18 is invoked, thus eventually leading to line 27. At this point, attackers can use reflection to call a no-argument `getter` method of `this._obj` (which points to the `JdbcRowSetImpl` object), leading to the `getDatabaseMetaData` method at line 30, and ultimately triggering a JNDI (Java Naming and Directory Interface) injection [20] at line 35. We use this example to illustrate the limitations of existing approaches and to highlight the key insights behind our proposed method.

Fake Edges in Call Graphs. Exploiting Java deserialization vulnerabilities often leverage the runtime polymorphic mechanism of Java [10], as shown in Figure 1 and Figure 2. To tackle this, current detection approaches rely on Class Hierarchy Analysis (CHA) [13] to construct call graphs, which computes all overridden methods in subclasses to cover potential paths during deserialization. However, relying solely on CHA may introduce unreachable fake edges, resulting in a significantly large and redundant graph (known as the path explosion problem [40]). This not only limits the scalability of vulnerability detection, but also leads to false positives due to the over-approximation of return values. For example, existing tools like TABBY [12] will resolve 20 callees at line 6 in Figure 2, among which 19 callees are fake call edges.

Pointer analysis offers a more precise way to build the call graph by determining callees based on the objects referenced by the receiver variable. However, simply adopting pointer analysis introduces several critical issues. First, during deserialization, we must consider the polymorphism caused by controllable objects, as well as resolving callees by the `new` objects. Addressing these two aspects collectively is challenging since there is flow of both `new` and controllable objects during deserialization. Second, pointer analysis will induce heavy overhead since current applications often rely on plenty of Java libraries, greatly increasing the analysis scope (e.g., a simple SpringBoot application includes 71 libraries).

To address the above challenges, our key insight is to leverage a novel *demand-driven pointer analysis* [47] considering deserialization features on necessary variables, and then adopt hybrid dispatch based on the analysis results. For example, at line 5 in Figure 2, where `this.deserializer` is initialized as a `MapDeserializer` object, we can determine the unique callee at line 6 if our analysis tells the points-to relationship. At line 16 where our analysis determines that `this._obj` points to a controllable object, then we can resolve the callees

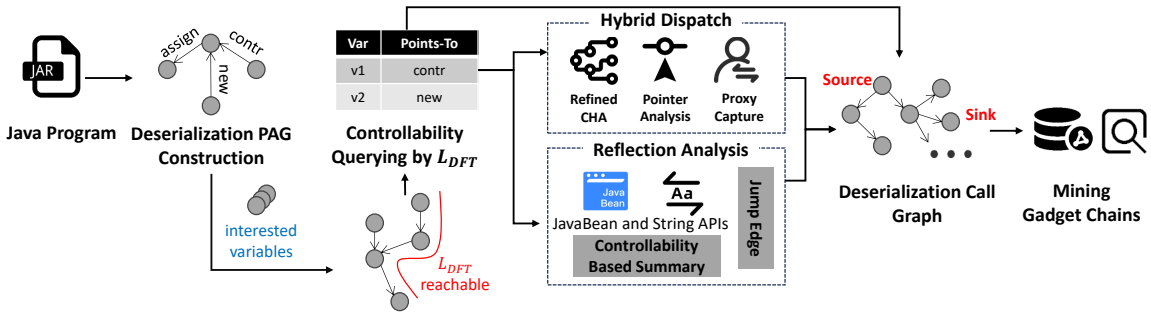

Figure 3: The overall workflow of FLASH

based on CHA to handle polymorphism. Such a strategy can enhance efficiency while guaranteeing precision.

Missing Edges in Call Graphs. While certain existing approaches have considered the polymorphic mechanism when constructing call graphs, they lack sufficient support for advanced dynamic features such as reflection and dynamic proxy. For example, at line 27 in Figure 2, the reflection `invoke` will be simply treated as the sink method by current tools [9–12, 28]. As a result, call graphs will lack runtime paths, causing the detection algorithm to overlook subsequent gadgets. In this example, it acts as a link gadget directing execution to the `getDatabaseMetaData` method at line 30. Without reflection, the attacker cannot reach the actual sink `lookup` at line 35 and thus cannot complete the exploitation. In addition, attackers can use dynamic proxy to direct the deserialization process to handler methods by proxy objects when crafting exploits (as noted in Section 2.3). Therefore, there may be more hidden targets at call sites during the deserialization process. Neglecting these features will result in missing edges in the call graphs, thus eventually leading to false negatives during gadget chain detection.

Unfortunately, performing reflection and dynamic proxy analysis are also challenging. First, Java reflection is known to be an obstacle for static analysis [26], especially in deserialization scenarios where controllable fields may not be assigned values explicitly. Therefore, it is crucial to consider controllability when collecting feasible call targets. As for dynamic proxy usage, there is no research considering this aspect in detecting JDVs to our best knowledge. Instead, they are simply considered as trampoline methods (common link gadgets [48]). As noted in Section 2.3, the assumption that all controllable call sites can be proxied would result in numerous false positives.

Our solution for handling dynamic proxy is to treat it as a special form of method dispatch, allowing us to model their behaviors based on static analysis. In particular, we manually collect features to determine whether the receiver variable of a call site can be proxied and add *proxy jump edges* correspondingly. For reflection, our solution is to integrate controllability analysis to model Java's built-in string [44] and JavaBean APIs to infer reflection targets. Then we also add *reflection jump edges* to the call graph to make it more sound and facilitate the automated detection of gadget chains. For example, on line 26 where we identify that variable $pds[i]$ is controllable and the method `getReadMethod` contains summaries of JavaBean manipulation, we can conclude that the controllability value of variable *pReadMethod* begins with '`get`' and then infer feasible callees. By modeling JDK built-in APIs, we can effectively analyze an application, even if it encapsulates these APIs into new class methods.

3 Approach Design

3.1 Overview

We propose FLASH, an approach focused on constructing precise call graphs efficiently for gadget chains mining based on DDCA (Deserialization-Driven Controllability Analysis). Figure 3 shows the overview of FLASH, which contains the following main steps. First, FLASH performs DDCA on variables of interest for determining whether a variable can be restored via deserialization. Leveraging the outcomes, FLASH then proceeds to execute a range of downstream tasks (e.g., hybrid dispatch and controllability based reflection analysis) and makes the best-efforts to construct the deserialization call graph (i.e., call graph considering deserialization context features), and based on which FLASH detects gadget chains.

3.2 Controllability Analysis

3.2.1 DPAG Construction

DPAG extends the traditional Pointer Assignment Graph (PAG) [47, 49, 54] by introducing novel `contr` *edges* and *controllable objects*, alongside the standard pointer manipulation edges for five types of Java statements: `new`, `assign`, `cast`, `load`, and `store`. For the `call` statement, we handle it by applying method summaries similar to an existing study [12]. FLASH assumes that all parameters of a method under analysis, including the `this` variable, are initially controllable. For each parameter p, it creates a controllable object o_c and adds the edge $o_c \xrightarrow{\text{contr}} p$. Variables generated by `new` statements

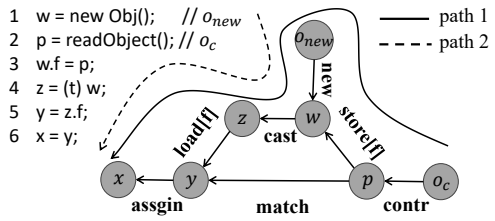

Figure 4: The DPAG of a code snippet, with inverse edges omitted for clarity.

are deemed uncontrollable, while the controllability of other variables (e.g., local variables) are determined through DDCA queries (see Section 3.2.3). To ensure field sensitivity, FLASH incorporates `match` edges into DPAG, following existing studies [43,47]. These `match` edges connect the source variable of the `store` statement to the target variable of `load` statement accessing the same field. By iteratively refining these match edges (see Algorithm 1), FLASH determines whether the base variables form aliases. Finally, for each DPAG edge $v \xrightarrow{l} v'$, its inverse edge $v' \xrightarrow{\bar{l}} v$ is also included in DPAG.

Figure 4 uses a code snippet to illustrate the corresponding DPAG. Since the value of p is obtained from `readObject()`, a deserialization method, the edge $o_c \xrightarrow{\text{contr}} p$ is added to DPAG. Here, p and y serve as the source and target variables of `w.f=p` and `y=z.f`, respectively, and thus a `match` edge is added between p and y.

3.2.2 The Design of L_{DFT}

As noted in Section 2.2, demand-driven pointer analysis is typically solved based on the CFL-reachability. Therefore, in addition to the graph representation, a context-free grammar specific to deserialization is also required. Figure 5 presents the grammar of L_{DFT} designed for filtering infeasible object flow paths in the DPAG, which extends existing works [46,47], with our main contribution highlighted in blue.

The production rule $DFT \rightarrow contr\ (load[f])$ accounts for the deserialization feature, ensuring that if a variable is controllable, its fields are also considered controllable. However, variables marked with the `transient` keyword are exempt from this mechanism, and FLASH handles them with special care. When analyzing a method, FLASH first checks whether the deserialization-related methods (e.g., `readObject`, `readExternal`) of its declaring class

$$
\begin{aligned}
DFT &\rightarrow new \mid contr\ (load\ [f]) \\
DFT &\rightarrow DFT\ (assign \mid cast) \\
DFT &\rightarrow DFT\ store[f]\ alias\ load\ [f] \\
alias &\rightarrow \overline{DFT}\ DFT
\end{aligned}
$$

Figure 5: The context free grammar for L_{DFT}

have been analyzed. If not, it prioritizes their analysis to recover `transient` variable values and reduce false positives. The second rule indicates that objects continue to flow when passing through `assign` or `cast` edges. The third rule incorporates Java's field-sensitivity. The last rule computes alias relationships, where $\overline{DFT}$ denotes the points-to relationship.

3.2.3 Controllability Querying with L_{DFT}

Before presenting the algorithm to address controllability queries, we employ the following strategies to automatically gather a set of variables of interest:

- **Call site variables.** All variables at call sites are targeted, as comprehensive information about call sites is essential for constructing the call graph.
- **Method summary variables.** Following a recent study [12], variables such as method parameters and return values are also targeted, as they aid in constructing method summaries to eliminate redundant analysis and facilitate inter-procedural analysis.

Section 4 shows that this subset of variables is sufficient for detecting gadget chains effectively.

Algorithm 1 answers FLASH's on-demand controllability queries for target variables by resolving DPAG reachability using L_{DFT} in intra-procedural analysis. Built upon the existing algorithm [47], it is adapted for deserialization scenarios with our contributions highlighted in gray. At line 6, Algorithm 1 leverages caching to avoid redundant queries. At line 8, a set is used to prevent infinite recursion. For the newly introduced `contr` edges, we handle them similarly to the `new` edges. As shown in lines 13–14, the source object of `contr` edge is added to the points-to set. Additionally, we support the handling of `cast` edges to improve precision. In lines 17–23, Algorithm 1 first retrieves the points-to information of the source variable. If the source object is controllable, its type is updated to enable more precise resolution of callees. If the object is not controllable, the algorithm checks the cast condition before adding it to the result. In lines 24–34, the algorithm first performs field-sensitive analysis based on `match` edges. If a field has no `match` edge and its base variable is controllable (line 32), the algorithm will set the field as controllable, according to our introduced production rule as shown in Figure 5. Finally, in line 35, the computed information is cached. The remaining parts of the algorithm follow prior work [47] and primarily ensure the object flow and field sensitivity in Java. When multiple objects can flow into a variable, FLASH merges the results in the following situations: (1) if the objects involve `new` and controllable objects, FLASH considers the variable is controllable. (2) if the objects are all controllable, FLASH favors the object with the upper type for over-approximation.

We can derive the points-to information from the perspective of L_{DFT}. The following equation outlines the derivation

Algorithm 1: Query for querying points-to-set of interested variables

```
   Input: variable v
   Output: points-to-set of v
1  global cache mapping c;
2  set s ← ();
3  pointsTo ← query(v, s);
4  return pointsTo;
5  Function query(p, s):
6      if p in c then return c[p] ;
7      pointsTo ← (); workList ← []; visited ← ();
8      if p in s then return pointsTo ;
9      add p to s ;
10     propagete(p, visited, workList);
11     while workList is not empty do
12         remove x in front of workList;
13         for new, contr edge o → x do
14             add o to pointsTo;
15         for assign edge y → x do
16             propagate(y, visited, workList);
17         for cast edge e = y -t-> x do
18             ptOfFrom ← query(y, s) ;
19             for obj in ptOfFrom do
20                 if obj is controllable then
21                     set type t for obj; add obj to pointsTo ;
22                 else if typeof(obj) <: t then
23                     add obj to pointsTo ;
24         for loadField[f] edge e = p → x do
25             matchEdges ← pag.getMatchEdges(f);
26             found ← false ; pt_P ← query(p, s);
27             for storeField[f] edge y → q in matchEdges do
28                 pt_Q ← query(q, s);
29                 if pt_P intersects pt_Q then
30                     found ← true ;
31                     propogate(y, visited, workList);
32             if found is false and pt_P is controllable then
33                 get controllable object o from pt_P ;
34                 add o to pointsTo ;
35     c ← {p: pointsTo} ;
36     return pointsTo;
37 Function propagate(p, visited, workList):
38     if p not in visited then
39         add p to visited and workList;
```

process of variable x pointing to o_c (with intermediate nodes omitted), which is also equivalent to the analysis process of Algorithm 1. However, for path 2 which does not satisfy the grammar, we cannot conclude that x points to o_{new}. Therefore, Algorithm 1 will not output this result.

$$
\begin{aligned}
& o_c\ contr\ store[f]\ \overline{new}\ new\ cast\ load[f]\ assign\ x \\
\rightarrow\ & o_c\ DFT\ store[f]\ \overline{DFT}\ DFT\ load[f]\ assign\ x \\
\rightarrow\ & o_c\ DFT\ store[f]\ alias\ load[f]\ assign\ x \\
\rightarrow\ & o_c\ DFT\ assign\ x \\
\rightarrow\ & o_c\ DFT\ x
\end{aligned}
$$

For inter-procedural analysis, after analyzing a callee at a call site FLASH applies its summary for side effect analysis. Similar to the approach in a related study [12], FLASH queries the controllability of all method summary variables after analyzing the method and represents their influence on function parameters and return variables in the form of a map. Subsequently, FLASH updates the variables at the call site based on the action [12] outlined in the summary map.

After obtaining the controllability results of the interested variables, FLASH leverages the information to perform downstream tasks such as method dispatch and reflection inference, thus efficiently constructing a more precise call graph.

3.3 Hybrid Dispatch

We propose a hybrid dispatch strategy for callee resolution to tackle challenges posed by deserialization-induced runtime polymorphism and dynamic proxy.

For a call site, we first use Algorithm 1 to check the receiver's controllability. If the receiver is uncontrollable (e.g., points to a `new` statement object), indicating no runtime polymorphism at the call site, FLASH employs pointer analysis to resolve callees with high precision. If the receiver is controllable (e.g., points to a controllable object), FLASH uses CHA to identify all overridden callees, ensuring soundness for runtime polymorphism. It further refines the results by filtering out unusable callees based on specific strategies built on existing research [40, 41], while enhancing soundness in deserialization scenarios. Section 4.3 demonstrates that they do not affect the false negative rate.

- **Serializable.** A callee is ignored if its declaring class does not implement `java.io.Serializable`. FLASH can be configured to bypass this strategy if the application uses a customized deserialization mechanism.
- **Visibility.** A callee is ignored if it cannot be accessed externally per Java's accessibility rules [33]. For instance, if the callee is `private` and the call is a normal invocation (e.g., not via reflection), Java's rules prevent invocation, even if we control an object of the callee's class.
- **Subtype.** Considering type casting, a callee is ignored if its declaring class is not a subtype of the controllable object pointed to by the receiver variable.

Next, FLASH checks if a call site can trigger dynamic proxy behaviors. If so, it adds the corresponding handler methods to the dispatch result set. As noted in Section 2.3, not all controllable call sites trigger dynamic proxy behaviors as binding receiver variables to proxy objects may cause exceptions during the payload generation or deserialization phase. Therefore, we systematically collected dynamic proxy features, as illustrated in Figure 6. First, we thoroughly reviewed the official JDK documentation on dynamic proxy [34]. Next, we collected as many publicly available proof-of-concepts (POCs) involving dynamic proxy as possible, including reviewing detection results from related research [8–12, 17, 21, 28, 30, 35, 48, 53], searching GitHub repositories and CVE reports, from which we summarized additional features. We then manually constructed POCs based on the identified features to trigger dynamic proxy, and observed whether exceptions occurred during serialization or deserialization to validate the features.

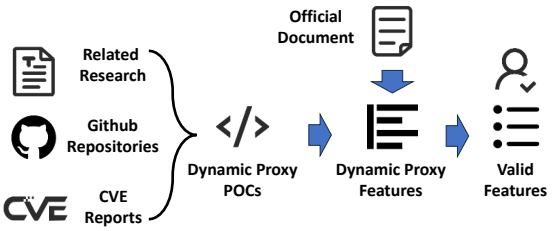

Figure 6: The collection pipeline for dynamic proxy features

This pipeline took approximately one week for two security experts familiar with JDVs to complete, resulting in the following features. In particular, a call site triggers a dynamic proxy only if all of the following features are satisfied:

- **Proxy Implementation.** The application containing the call statement must include a proxy implementation, specifically an application class that implements both *java.io.Serializable* (or a custom deserialization mechanism) and *java.lang.reflect.InvocationHandler*.

- **Interface Call.** According to [34], the call site statement must be an `interface call` to trigger proxy dispatch; Otherwise, a `ClassCastException` will interrupt the payload generation process.

- **No Cast.** The objects pointed to by the receiver variable should not be cast. Since proxy objects inherit from *java.lang.reflect.Proxy*, any cast will throw a `ClassCastException`, interrupting deserialization.

Once the proxy dispatch target is identified, we introduce dynamic proxy jump edges to model the unique parameter propagation and ensure accurate controllability tracking. As illustrated in lines 1-14 of Figure 7, `invoke` serves as the dynamic proxy target for the call `this.field.callee`. The receiver variable (e.g., `this.field`) is passed to both the `this` variable and the first parameter, `proxy`, of the `invoke` method. The second parameter originates from the call site and is considered uncontrollable, while the third parameter, `arg`, encapsulates all call-site arguments into an array. To prevent false negatives, we assume that if any element in the array is controllable, the entire array is considered controllable. Following prior work [11], we use a method route table constructed through path-sensitive analysis to manage the handler methods within the proxy method. For instance, if the proxy method (line 5) in Figure 7 is triggered by a `hashCode` call, it is routed to the `hashCodeImpl` method.

In conclusion, FLASH uses PA to handle dispatch for call sites with uncontrollable receivers, reducing false edges. For controllable receivers, it accounts for deserialization-induced polymorphism and dynamic proxy scenarios, minimizing missing call edges. This hybrid dispatch enables more precise construction of the deserialization call graph.

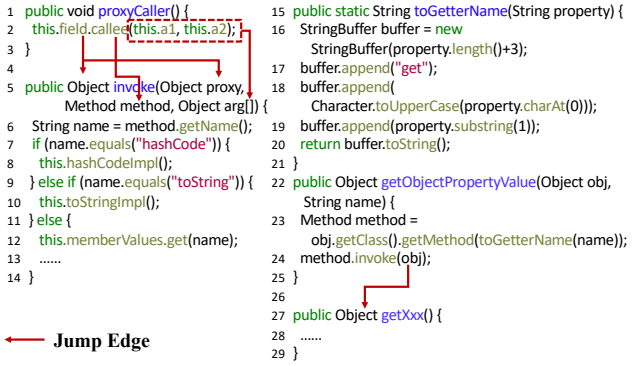

Figure 7: Add jump edges for dynamic proxy and reflection

3.4 Controllability Based Reflection Analysis

Our controllability-based reflection analysis focuses exclusively on three gadget-linking APIs (`Method#invoke`, `Constructor#newInstance`, and `Class#forName`), while classifying all other reflection APIs as either sinks or controllability transfers [22]. For these three APIs, when invocations involve controllable arguments, we utilize existing techniques [25–27] to resolve potential callees. FLASH further refines these results based on the number, type and controllability of additional parameters, introducing reflection `jump` edges to the call graph to propagate controllability and enable effective gadget chain detection.

To facilitate controllability propagation to reflection-related APIs, we manually model a set of 72 JDK built-in string manipulation APIs inspired by prior work [44] that manually models string operations for reflection analysis. Following similar process of Figure 6 to collect reflection-related POCs, we additionally observe the widespread usage of JavaBean operations and also model 10 JavaBean property manipulation APIs in the JDK. The above process took approximately one week for the two security experts to finish. Our approach focuses on creating summaries specifically for controllability propagation. For instance, if a string slicing operation involves a controllable variable, our summary ensures the result is also marked as controllable. Beyond simple transfers, we address various forms of controllability, such as concatenation, by modeling additional summaries for diverse operations. By integrating controllability analysis, our approach simplifies string and JavaBean manipulations while ensuring effective propagation of controllability. Note that all modeled APIs and methods are open-sourced on our website, and users can refer to the documentation corresponding to their experimental environment. To ensure consistency with the latest JDK APIs, it is only required to check JDK update notes for changes to the relevant APIs (String and JavaBean manipulations). We are also currently working on collecting and modeling built-in string and JavaBean APIs across all JDK versions to enable users to run our tool by only specifying the JDK

version, thereby minimizing the additional effort.

Lines 15-29 of Figure 7 illustrate how FLASH handles reflection. Specifically, the `getObjectPropertyValue` method passes the controllable variable *name* to `toGetterName`, which uses a `StringBuffer` to return a partially controllable string. In the first `append` call, FLASH assigns '`get`' as the controllability value of *buffer*. During the second `append` call, since variable *property* is controllable, FLASH applies the method summary for `toUpperCase` to generate a controllable value '`param-0`' and concatenates with '`get`' to derive the final controllability value '`get+param-0`'. As in the third `append` call, *buffer* is already partially controllable, FLASH avoids further concatenation. Therefore, the final controllability value '`get+param-0`' is propagated to the variable *method* in `getObjectPropertyValue` via the `toString` and `getMethod` calls, enabling the resolution of callees (i.e., methods starting with '`get`' and having no parameters) for the `invoke` call. FLASH then adds jump edges to link these callees and matches parameters, addressing missing call graph edges and preventing false negatives in gadget chain detection. FLASH applies a similar approach for the other two types of reflection APIs, with the distinction that their method names are fixed as '`init`' or '`clinit`', enabling inference based solely on parameters.

3.5 Gadget Chain Detection

After constructing the deserialization call graph, FLASH applies existing algorithms [12] for gadget chain detection. Simply put, FLASH performs a backward analysis from the sink to determine whether any controllable variables flow to the sink. However, since this paper introduces two new types of call edges (proxy and reflection jumping edges), additional filtering is performed to eliminate false positives. For reflection, FLASH also performs a backward analysis from the method name variable to identify which variable (say $g1$) influences its controllability. Then it verifies whether the name of the callee matches the controllability of $g1$. For dynamic proxy, Flash analyzes based on the method route table as mentioned in Section 3.3. For example, if $g2$ triggers the dynamic proxy handler method h via an interface call `hashCode`, based on the method route table, FLASH determines that h should route to `hashCodeImpl`, and other callees are false positives.

4 Evaluation

To evaluate the effectiveness and usefulness of FLASH, we design the following four main questions. We also provide additional analysis in Appendix A.2 on the other benefits that FLASH offers to security researchers.

RQ1. Effectiveness. How effective is FLASH in detecting gadget chains based on the deserialization call graph?

RQ2. Ablation Study. How do FLASH's key components contribute to the performance of gadget chains detection?

RQ3. Efficiency. How efficient is FLASH in constructing the deserialization call graph?

RQ4. Vulnerability Analysis. What is the security risk of the detected gadget chains?

4.1 Setup

Benchmark. We utilize the dataset used by TABBY, and further manually collect Java libraries that exhibit gadget chains as additional examples. In Table 1, we use a horizontal line to separate the results from the dataset used by TABBY and those newly collected in our study. To improve the completeness, representativeness, and timeliness of our dataset, we follow a two-step selection process. First, we filter all pull requests from *ysoserial* [15] and *marshalsec* [29], as they are the source of current datasets and more likely to contain additional gadget chain information. We select gadget chain disclosures submitted within the past three years and focus on applications absent in existing datasets, resulting in the inclusion of WildFly and SnakeYAML. Second, we examine the top 10 applications from each of the top 100 Maven repository categories, identifying those with publicly disclosed gadget chains within the same time frame. This lead to the addition of XStream, Dubbo, Fastjson, and Jackson. In total, the benchmark includes 30 libraries and 56 known gadget chains.

Baselines. Currently, gadget chain detection tools can be categorized into static analysis approaches [8, 12, 17, 28, 53] and hybrid ones [9, 10, 21, 35, 44, 48] (with fuzzing modules). We choose TABBY [12] from static approaches, and CRYSTALLIZER [48] and JDD [11] from hybrid ones since they are the state-of-the-art and open-sourced. We do not compare with GADGETINSPECTOR [17] and SERIANALYZER [35] since their performance is inferior as reported by all the three selected baselines. We exclude ODDFUZZ [9] as its performance is inferior to that of JDD [11] as reported by the study. We exclude REVGADGET [28], GCMINER [10], GCGM [21] and HAWKGADGET [53] as their source code are not available. Therefore, the experiments compare three state-of-the-art tools. Note that the fuzzing component of JDD is not open-sourced [3], therefore we implemented a prototype system based on the description provided in the paper, achieving similar performance by comparing the results with those in the publicly released dataset of JDD.

Sink Selection. Building on existing research [9, 12], we have collected 28 sinks, all of which are open-source. Users can also easily extend the sink configuration for detection.

Gadget Chain De-Duplication and Verification. Current tools such as TABBY consider that two gadget chains gc_1 and gc_2 are distinct if they contain at least one different gadget. This strategy can result in a considerable number of gadget chains due to the inherent flexibility and variety of gadget chain combinations. In this study, we introduce a more rigorous strategy to de-duplicate chains by considering gadget chains with identical source-sink pairs as equivalent. The ra-

Table 1: Detected gadget chains of FLASH and the other baselines

Java Library[1]	Know[2]	Tabby				Crystallizer				JDD				Flash			
		All[3]	DD[4]	Cov[5]	New[6]	All	DD	Cov	New	All (F[7])	DD	Cov	New	All	DD	Cov	New
AspectJ[8]	1	5	5	1	2	85	3	1	2	16 (6)	2	0	2	335	10	1	4
Bsh[9]	1	14	1	0	0	4	0	0	1	0 (0)	0	0	0	1,238	20	1	12
C3P0	1	139	8	1	5	0	0	0	0	15 (0)	0	0	0	6	6	1	5
Click	1	123	4	0	0	0	0	0	0	34 (1)	1	1	0	15	2	1	1
Clojure	1	0	0	0	0	0	0	0	0	169 (3)	3	1	2	1	1	0	1
CB[10]	1	786	11	0	0	2	2	0	1	9 (1)	1	1	0	84	6	1	0
CC3[11]	5	3,381	27	4	7	80	49	3	3	196 (53)	6	3	3	2,665	44	5	21
CC4[12]	2	2,746	22	2	6	4	4	2	2	209 (51)	8	1	7	1,863	40	2	15
CC[13]	1	0	0	0	0	0	0	0	0	157 (0)	0	0	0	80	8	0	4
File[14]	2	107	2	2	0	0	0	0	0	1 (1)	1	1	0	2	2	2	0
Groovy	1	719	3	0	0	7	7	0	1	580 (7)	5	1	4	0	0	0	0
Hiber[15]	2	1,875	6	2	0	0	0	0	0	0 (0)	0	0	0	0	0	0	0
Jassist[16]	1	19	4	1	0	0	0	0	0	2 (1)	1	1	0	20	1	1	0
JBoss	1	0	0	0	0	0	0	0	0	2 (1)	1	1	0	10	1	1	0
JSON	1	0	0	0	0	0	0	0	0	0 (0)	0	0	0	0	0	0	0
Jython	1	204	6	0	0	0	0	0	0	0 (0)	0	0	0	1	1	0	0
Mozilla[17]	2	0	0	0	0	6	2	0	0	10 (1)	1	1	0	33	3	1	0
Myface	1	109	9	1	4	0	0	0	0	283 (3)	3	1	2	131	20	1	7
Resin	1	0	0	0	0	0	0	0	0	853 (5)	2	0	2	37	4	1	0
Rome	1	3	3	1	1	8	4	1	1	334 (9)	6	1	5	886	25	1	9
Spring	6	0	0	0	0	0	0	0	0	188 (0)	0	0	0	1,605	13	4	1
Vaadin	1	1,386	2	1	0	25	8	1	1	753 (39)	4	1	3	68	8	1	4
Wicket	2	151	3	2	0	0	0	0	0	1 (1)	1	1	0	2	2	2	0
XBean	1	162	3	1	0	0	0	0	0	421 (4)	2	1	1	2	2	1	1
Dubbo	4	218	7	2	0	11	4	0	0	31 (5)	3	2	1	32	8	2	2
FastJson	3	30	8	2	0	4	2	0	0	10 (3)	3	1	2	24	7	2	1
Jackson	3	63	10	2	0	0	0	0	0	10 (3)	3	1	2	17	6	1	1
SYaml[18]	2	181	6	0	0	0	0	0	0	39 (0)	0	0	0	17	5	2	0
Wildfly	1	10	2	1	0	0	0	0	0	4 (1)	1	1	1	1	1	1	0
XStream	5	498	14	4	0	2	2	0	0	150 (0)	0	0	0	78	8	2	1
Total	56	12,929	166	30	25	238	87	8	12	4,477 (199)	58	22	35	9,253	254	38	90

[1]The horizontal line separates TABBY's dataset from our newly added dataset; [2]Known gadget chain; [3]All detected gadget chains; [4]De-Duplication to all detected gadget chains; [5]Covered known chains; [6]Newly detected chains. [7] F: denotes confirmed by fuzzing; [8]AspectJ for AspectJweaver; [9]Bsh for BeanShell; [10]0CB for CommonsBeanutils; [11]CC3 for CommonsCollections3; [12]CC4 for CommonsCollections4; [13]CC for CommonsConfiguration; [14]File for FileUpload; [15]Hiber for Hibernate; [16]Jassist for JavassistWeld; [17]Mozilla for MozillaRhino; [18]SYaml for SnakeYaml;

Table 2: FNR and FPR of FLASH and the other baselines

Tool	Total All+DD	Total Cover	Total Fake	FNR	FPR
TABBY	166	30	111	46.4%	66.9%
CRYSTALLIZER	87	8	67	85.7%	77.0%
JDD	199	22	-	60.7%	-
FLASH	254	38	126	32.1%	49.6%

tionale behind this strategy is that different sinks often lead to different attacks while distinct sources signify different triggering paths based on known gadget chain analyses. Consequently, we prioritize the identification of different source-sink pairs in gadget chains, rather than focusing on the variations in intermediate paths. In this paper, if a gadget chain shares a different source-sink pair than another gadget chain, it is considered as a ***new gadget chain***. For detected gadget chains, we validate them via manual verification by two security experts well-versed in JDVs. If an attack payload can be successfully crafted based on a gadget chain and triggers the sink after being deserialized, then the gadget chain is confirmed to be exploitable, and we mark it as a *true positive*.

Hyperparameters. We set the maximum chain length to 12 for all experiments to ensure consistent evaluation, as over 95% of known gadget chains in the benchmark are no longer than 12. When reproducing CRYSTALLIZER, we run it with the recommended parameters, which are 1 hour for sink identification and 24 hours for fuzzing. When reproducing JDD, we follow its default configuration.

Environment. The host machine for the experiments is equipped with an Intel(R) Core(TM) i5-10400 2.9GHZ processor, 40 GB of RAM, and the JDK 8u20.

4.2 RQ1: Gadget Chain Detection Analysis

In this subsection, we primarily analyze the performance of FLASH in detecting deserialization gadget chains.

Table 1 illustrates the main results of gadget chain detection across various tools applied to the benchmark. It shows that each tool can detect a substantial number of gadget chains while there also exist many duplicated ones via comparing '`All`' and '`All-D`'. Taking `Click` as an example, after analyzing the results of TABBY manually, we discover that out of the 123 gadget chains, 119 contain redundant paths, which further reflect the complexity of various gadget chain combinations. Therefore, we use the results after de-duplication as the final outcomes for each tool.

Table 1 shows that FLASH outperforms all baselines and achieves the best performance. For distinct gadget chains,

FLASH detects in total 254 ones, surpassing the best baseline (i.e., TABBY) by 88 (i.e., 254-166). For covering known gadget chains, FLASH detects 38 out of 56 in the benchmark, identifying 8 more gadget chains than TABBY. Additionally, for new gadget chains, FLASH also uncovers the most with 90 gadget chains, exceeding the SOTA by 55.

4.2.1 False Negative and Positive Analysis

Table 2 presents the false positive rate (FPR) and false negative rate (FNR) for each tool. The '`Total Fake`' column denotes the total false positive gadget chains. The '`FNR`' column is calculated by tc/tk, where tc is the number of the total covered chains and tk is the number of total known gadget chains. The '`FPR`' column is calculated by tf/tdd, where tf is the number of total fake gadget chains and tdd is the number of total de-duplicated gadget chains. Since JDD directly uses fuzzing to generate exploitable results, it does not produce false positives. Therefore, the corresponding `Total Fake` and `FPR` are excluded in Table 2. Table 2 shows that FLASH exhibits the lowest FNR of 32.1% and lowest FPR of 49.6% (excluding JDD). Compared to the best baseline, FLASH reduces the FNR by **30.8%**(=(46.4%-32.1%)/46.4%), and FPR by **25.9%**(=(66.9%-49.6%)/66.9%) (excluding JDD), respectively, demonstrating its superior performance.

The lower FNR of FLASH can be attributed to more sound call graphs obtained through the support of analyzing dynamic proxy or reflection usages. For example, Spring's known gadget chains are highly complex, with some even employing triple dynamic proxy techniques, causing existing baselines fail to cover any known gadget chains. However, thanks to FLASH's hybrid dispatch and dynamic proxy jump edges, it can successfully cover many gadget chains. FLASH's lower FPR stems from two main factors: the integration of PA in hybrid dispatch, which improves deserialization call graph precision, and effective jump edge construction and filtering strategies that prevent false positives significantly. Although JDD leverages fuzzing to directly produce exploitable results and therefore does not generate false positives, it exhibits a higher FNR and detects fewer new gadget chains compared to FLASH. We illustrate such limitations with examples of chains that fuzzing fails to cover in the case study provided in Appendix A.1. On the contrary, FLASH prioritizes reducing

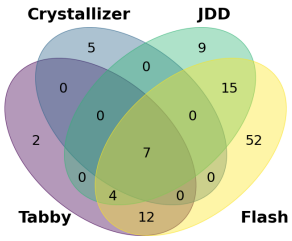

Figure 8: The overlap of the detection results of new chains

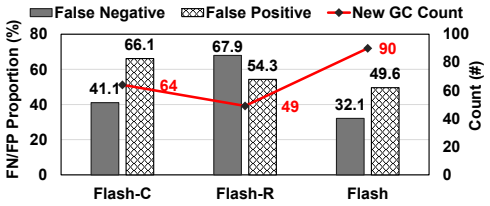

Figure 9: Comparison of FLASH-C, FLASH-R and FLASH

false negatives, which is critical for vulnerability detection, thus achieving a rate of only 32.1%.

In conclusion, the lower FNR demonstrates that the deserialization call graph used by FLASH is more sound and the lower FPR demonstrates that the deserialization call graph used by FLASH is more precise, thereby enhancing its performance of gadget chain detection.

4.2.2 New Gadget Chains Detection Analysis

A new gadget chain represents a previously unknown path to trigger methods containing malicious behaviors. If developers do not impose restrictions on the gadgets involved, it can easily lead to JDV risks. Table 1 shows that FLASH detects a total of 90 new gadget chains, while TABBY, CRYSTALLIZER and JDD detects 25, 12, and 35 new ones, respectively. This highlights FLASH's superior performance in discovering new chains. In particular, FLASH not only discovers new gadget chains in applications where TABBY, CRYSTALLIZER, and JDD fail, such as SnakeYaml, but also identifies more new gadget chains in applications where TABBY, CRYSTALLIZER, and JDD can already detect, such as CC3, CC4 and Rome.

We further investigate the distributions of the new gadget chains discovered by all the tools, with the results illustrated in Figure 8. Among the 25 new gadget chains discovered by TABBY, FLASH successfully covers 23 of them. The two uncovered gadget chains are due to FLASH not considering the detection of secondary deserialization (read controllable data and perform deserialization after source). As for CRYSTALLIZER, apart from the 5 gadget chains it identifies that lead to DoS attacks, FLASH successfully covers the remaining new chains. Compared to JDD, FLASH also demonstrates a substantial improvement in the number of new chains detected. FLASH detects fewer gadget chains in Groovy compared to JDD as it does not include a special sink used by JDD. For Clojure, the reason is its largest call graph among all applications as shown in Table 3. Since FLASH focuses on call graph construction, the gadget chain detection algorithm is implemented based on TABBY, which may struggle to effectively identify gadget chains in Clojure. We'll expand our sink set and explore more efficient detection algorithms in the future.

The above results demonstrate that FLASH is more effective in detecting new chains, both in terms of quantity and quality.

Table 3: The call site distribution and scale of call graphs constructed based on different strategies (the unit is *k*).

Java Library	CS Distribution[1] PA[2]	DP[3]	All	Method CHA	CHA+P[4]	HD	Call Edge CHA	CHA+P	HD
AspectJ	64.2	15.2	263.6	76.0	76.1	75.5	856.9	870.4	794.3
Bsh	44.6	7.4	177.5	53.5	54.0	53.3	434.6	436.7	396.6
C3P0	28.9	4.2	110.8	34.6	34.8	30.1	376.1	379.4	312.9
Click	44.4	7.9	178.4	54.2	54.4	53.6	452.1	452.8	405.8
Clojure	51.8	9.1	248.0	71.4	72.9	66.3	2945.7	3102.7	2652.4
CB	44.3	7.8	177.5	54.3	54.4	53.2	451.3	451.8	403.5
CC3	55.9	12.2	229.0	68.2	68.3	67.8	674.5	679.8	630.5
CC4	54.7	12.6	228.0	67.8	67.9	67.5	636.0	654.7	578.1
CC	41.2	8.5	167.7	46.1	46.2	45.9	497.7	501.8	461.7
FileUp	27.7	3.7	106.1	30.1	30.3	29.6	275.3	294.2	247.7
Groovy	65.2	14.5	284.6	81.5	81.6	81.1	1367.3	1405.0	1319.2
Hiber	34.2	5.6	135.4	39.9	41.2	39.4	439.0	462.8	428.4
Jassist	32.2	4.9	124.6	38.7	38.8	36.7	338.7	346.0	304.1
JBoss	31.7	4.8	125.6	37.6	37.7	36.6	332.1	332.8	283.0
JSON	27.7	3.7	104.5	30.3	30.4	28.3	273.3	280.3	240.6
Jython	27.1	3.5	103.1	29.1	31.6	28.7	262.6	290.9	244.5
Mozilla	32.2	4.9	125.4	36.0	36.1	34.9	336.9	341.6	298.5
Myface	48.9	9.3	199.0	74.4	74.5	61.7	843.7	852.7	576.0
Resin	82.2	16.1	345.6	92.7	93.6	93.3	1591.0	1673.7	1580.1
Rome	32.5	4.7	124.6	35.5	35.6	33.1	340.4	348.8	334.2
Spring	65.9	17.5	276.7	78.6	78.7	78.5	1331.2	1411.2	1329.9
Vaadin	37.2	6.6	145.1	43.8	45.5	44.9	394.8	425.8	389.9
Wicket	30.0	4.3	115.4	32.7	32.8	31.9	297.9	342.4	267.6
XBean	40.7	8.4	166.2	45.6	45.9	44.7	478.9	501.3	450.9
Dubbo	58.3	14.1	243.3	70.1	71.2	69.7	792.0	879.6	729.7
FastJson	43.0	10.7	175.5	46.9	47.2	46.2	520.9	536.7	478.6
Jackson	41.1	8.2	167.0	47.1	47.5	46.3	517.7	538.7	487.5
SYaml	41.1	8.3	166.6	45.8	46.0	44.7	486.9	504.3	451.1
Wildfly	29.2	3.7	109.1	32.7	33.3	30.0	293.1	305.3	237.2
XStream	51.2	11.6	208.3	62.2	62.5	61.9	857.1	871.9	806.0
Average	43.6	8.5	177.7	51.9	52.4	50.5	657.0	682.5	604.0

[1]Distribution of call sites; [2]Number of call sites that can be resolved using pointer analysis;
[3]Number of call sites that can trigger dynamic proxying; [4]Considering dynamic proxying in CHA;

4.3 RQ2: Ablation Study

This section analyzes how FLASH's key components impact its overall effectiveness. Specifically, we implement FLASH-C and FLASH-R, where FLASH-C constructs the call graph simply based on CHA without hybrid dispatch and FLASH-R does not support reflection analysis. We then evaluate FLASH-C and FLASH-R on the benchmark under the same settings.

4.3.1 Hybrid Dispatch Analysis

Figure 9 shows that compared to FLASH, FLASH-C increases the FNR and FPR by 9.0% and 16.5%, reaching 41.1% and 66.1%, respectively. Besides, FLASH-C detects only 64 new gadget chains. This is mainly because: (1) Our hybrid dispatch module is capable of analyzing potential dynamic proxy handler callees, and its absence in FLASH-C leads to a higher FNR. (2) The integration of pointer analysis in hybrid dispatch enhances the precision of the constructed call graph, which in turn boosts the detection accuracy. Therefore, supporting Hybrid Dispatch enables FLASH to discover 26 new gadget chains (90 - 64) and reduce the FNR by 9%, indicating that 31 chains involve dynamic proxy (26 + 56 * 9%).

In order to gain a deeper understanding of the above results, we further investigate the constructed call graph, including the number of call sites that can be resolved using PA, the number of call sites that can trigger dynamic proxy, and the scale of the call graph under different strategies. Specifically, we design three call graph construction modules based on CHA, CHA+P (considering dynamic proxy in CHA), and hybrid dispatch respectively. The results are presented in Table 3.

Table 3 indicates that approximately 24.5% of call sites on average can be resolved more accurately by pointer analysis in hybrid dispatch and 8.5% of call sites can trigger dynamic proxy. This not only suggests that objects instantiated via `new` play a critical role in the deserialization process, but also emphasizes the significance of handling dynamic proxy. Relying solely on CHA for call graph construction can result in both fake and missing edges. Furthermore, by comparing the results in the '`CHA`' and '`CHA-P`' columns, we can observe that the call graph constructed based on CHA-P is larger (an increase of 0.5k methods and 25.5k call edges on average) due to the consideration of more handler methods. On top of this, the call graph constructed based on hybrid dispatch still maintains the smallest scale (2.7% reduction in methods and 8.1% in call edges compared to CHA), indicating that our hybrid dispatch can effectively improve the soundness and precision of the call graph. As a result, the results of FLASH exhibit lower FNR and FPR rates than FLASH-C. In addition, reducing the size of the call graph narrows the search space for gadget chain detection, thereby improving analysis efficiency.

In summary, constructing the call graph based on hybrid dispatch is more precise and sound than CHA, significantly enhancing the detection capability of gadget chains.

4.3.2 Reflection Support Analysis

We then analyze the contribution introduced by controllability based reflection analysis. Figure 9 shows that compared to FLASH, FLASH-R increases the FNR and FPR by 35.8% and 4.7%, reaching 67.9% and 54.3%, respectively. In addition, FLASH-R identifies only 49 out of FLASH's 90 new gadget chains. Therefore, supporting Reflection enables FLASH to discover 41 new gadget chains (90 - 49) and reduce the FNR by 35.8%, indicating that 61 chains involve reflection (41 + 56 * 35.8%). This demonstrates the significant importance of handling reflection for gadget chain detection.

In summary, the introduction of reflection analysis can further enhance the soundness of call graphs, thereby helping detect new gadget chains. A more sound call graph may also help fuzzing modules generate seeds that trigger reflection or dynamic proxy behaviors, ultimately reducing false negatives.

4.4 RQ3: Efficiency Analysis

Given that the primary focus and contribution of this paper lies in the construction of the deserialization call graph, this subsection evaluates FLASH's efficiency by comparing the time of the call graph construction. We chose TABBY and CRYSTALLIZER from the baseline methods due to their similar workflow, where a call graph is constructed before gadget chain detection. Note that based on the constructed call graph,

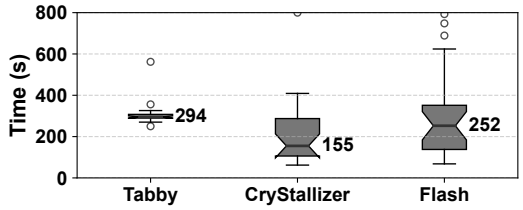

Figure 10: Time cost for constructing the call graph

various tools can apply similar algorithms (i.e., FLASH and TABBY use similar algorithms) for gadget chain detection, and thus the time consumptions are similar. Therefore, we only evaluate efficiency in terms of call graph construction, with the results presented in Figure 10.

Figure 10 shows that TABBY spends the most time in constructing the call graph on average (i.e., 294s), whereas CRYSTALLIZER and FLASH show a wider range of fluctuations. One possible reason is that TABBY constructs the call graph for the entire program, whereas CRYSTALLIZER and FLASH build call graphs starting from the deserialization entry points. Although CRYSTALLIZER uses the least time on average, its relatively higher FNR (see Section 4.2) suggests that its call graph construction may not be sound. In contrast, FLASH, even with the incorporation of pointer analysis, still constructs the call graph in a time comparable to CRYSTALLIZER and TABBY (and even better in some cases), proving the efficiency of FLASH's call graph construction. This result is also consistent with an existing study [6], which suggests that more precise analysis may yield better performance, as it may bypass the analysis of irrelevant code.

In addition, Section 4.3.1 has already shown that the call graph constructed by FLASH is smaller compared to that of CHA. This reduction in graph scale also leads to higher efficiency in subsequent gadget chain detection algorithms. In summary, both the process of constructing the call graph and the generated call graph demonstrate FLASH's high efficiency.

4.5 RQ4: Vulnerability Analysis

This subsection focuses on analyzing the security risks of the detected gadget chains, including their exploitability, FLASH's capability in identifying vulnerabilities, and the implications of new gadget chains.

Exploitability. The threat model of triggering JDVs often involves two stages accordingly to existing studies [11,12,42]: **Stage 1:** injection of malicious payload and **Stage 2:** successful execution of a gadget chain. **Stage 1** requires **Entry Existence** where the application can deserialize data and the critical factor in **Stage 2** is **Patch Omission** where the application has not patched the gadget chain either through code modifications or blacklist/whitelist mechanisms. The two conditions can prevent a gadget chain from being exploitable.

Figure 11 illustrates the core gadget of CVE-2023-23638 in

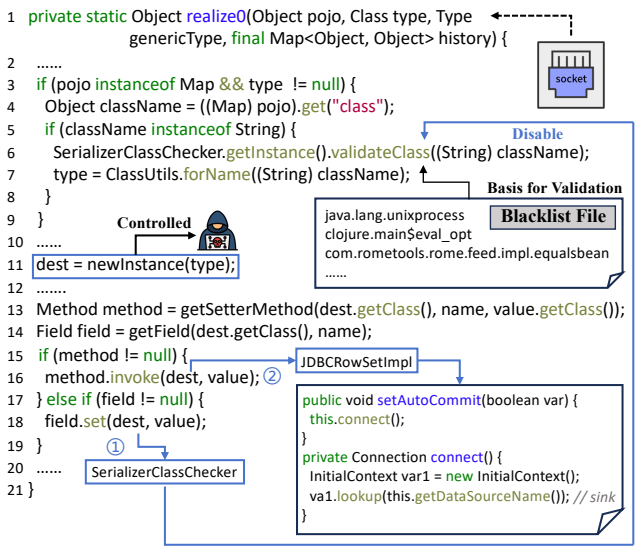

Figure 11: The core gadget chain of CVE-2023-23638

project Dubbo, which we use to explain the exploitability of a gadget chain. When Dubbo receives WebSocket data, it will deserialize the data and reach the `realize0` method, thereby satisfying **Entry Existence**. Within this function, although Dubbo applies a blacklist at line 6 to prevent deserialization of certain classes, an attacker can (1) first utilize the gadget to reach line 18, which can set the blacklist mechanism to be false and disable the blacklist patching mechanism. (2) Then the attacker can use reflection at line 16 to perform a JNDI injection. However, the gadget chain becomes unexploitable after JDK 8u113, as the JDK has been patched to restrict the loading of remote malicious classes.

FLASH's Capability in Identifying Vulnerabilities. Table 4 summarizes the number of vulnerabilities identified by FLASH that align with existing CVEs in our dataset, as well as newly discovered vulnerabilities. FLASH successfully detects 10 known CVEs. Leveraging the new gadget chains, FLASH uncovers 5 previously unknown exploitation methods of the vulnerabilities. Therefore, they can bypass existing patches and enable new types of attacks, showing a broader impact than previously known CVEs.

Implications of New Gadget Chains. Discovering new gadget chains enables us to find more new vulnerabilities, thereby expanding the attack surface. Take the CVE-2023-23628 in Figure 11 for example, note that the gadget chain becomes unexploitable after JDK 8u113. However, a new gadget chain from line 16 (`JEditorPane#setPage` → `JEditorPane#getStream` → `URL#openConnection`) discovered by FLASH can result in a SSRF attack across the entire JDK 8, resulting in a new vulnerability and a new type of attacks. In summary, uncovering new gadget chains not only facilitates vulnerability detection but also can contribute to the development of more comprehensive patches.

Table 4: Vulnerability detection of FLASH

Detected Vulnerabilities	Dubbo	FastJson	SnakeYaml	XStream
Known CVE	4	2	2	2
New	2	1	0	2

Based on the exploitability conditions and security implications, we further classify the gadget chains detected by FLASH to demonstrate the potential security risks. The results are illustrated in Figure 12. For the **Entry Existence**, 15 gadget chains are identified in libraries with deserialization entries, while 113 are found in libraries without such entries. For the latter, we manually generated entries to validate their exploitability, and find all of them to be exploitable. For **Patch Omission**, given that all prior researches [8–12, 17, 21] were conducted under JDK 8 and the Tabby study [12] also emphasized JDK 8's prevalence in real-world applications, we analyze the exploitability across different versions of JDK 8. Note that the classification approach is generalizable to other JDK versions, and we will explore this direction in future work. Specifically, 14 gadget chains are exploitable in versions prior to 8u71, 6 gadget chains are exploitable between 8u71 and 8u113, while 108 gadget chains remain exploitable in all JDK 8 versions. We also categorize the types of attacks lead by gadget chains according to their sink methods [12], eventually identifying a total of 7 distinct types.

5 Related Work

5.1 Gadget Chains Detection

In recent years, detecting gadget chains has gained significant attention in security research, leading to the development of several detection algorithms [8–12, 17, 21, 28, 30, 35, 48, 53]. Both *GadgetInspector* [17] and *Serianalyzer* [30] use taint analysis on Java bytecode to determine if controllable values propagate to sink methods. *SerHybrid* [35] is the first tool to attempt using fuzzing for gadget chain validation, which is based on based on heap access paths through static analysis. *GCGM* [21], an extension of *GadgetInspector*, incorporates symbolic execution for call graph generation and introduces dynamic verification. *JChainz* [8] introduces a Type Propagation Analysis through the Data Dependency Graph to reduce false positives. *Tabby* [12] performs controllability analysis

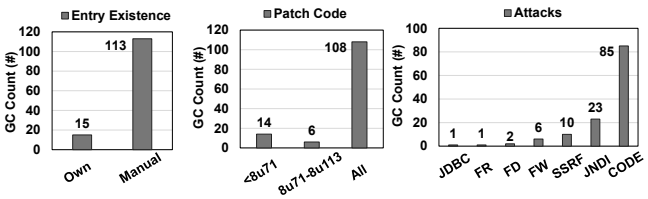

Figure 12: Security risk of gadget chains detected by FLASH

and introduces a Code Property Graph for customizable gadget chain queries. *GCMiner* [10] proposes Overriding-Guided fuzzing to generate exploitable objects, while *ODDFuzz* [9] enhances efficiency with Structure-Aware Directed Greybox fuzzing. *Crystallizer* [48] proposes the dynamic sink identification technique and detects gadget chains based on gadget graph. *RevGadget* [28] discovers gadget chains based on reverse semantics and taint analysis. *JDD* [11] improves the efficiency of searching gadget chains within the call graph by utilizing gadget fragments. In this study, we propose FLASH, a novel gadget chain detection tool leveraging a deserialization-guided call graph construction approach to address the fake and missing call edges in current approach.

5.2 Program Analysis Technique

Demand-Driven Pointer Analysis. Demand-driven pointer analysis focuses only on the pointer sets related to interested variables, striking an effective balance between precision and efficiency. Sridharan et al. [47] introduce CFL-reachability based on balanced parentheses to implement demand-driven pointer analysis for Java and improve efficiency using matched edges. Shang et al. [43] speed up the analysis by dynamically generating method summaries. Spath et al. [45] perform demand-driven flow- and context-sensitive pointer analysis for Java based on IFDS [39]. In this work, we develop L_{DFT} by adding `contr` edges and production rules, ensuring the flow of both new and controllable objects.

Reflection and Dynamic Proxy Analysis. Effectively handling features like reflection and dynamic proxy in static Java analysis is crucial. Li et al. [26] introduce Solar, a lazy heap model that infers the targets of reflection call sites and identifies difficult reflection points, seeking user feedback for refinement. Song et al. [44] enhance reflection analysis by modeling JDK string APIs and parsing configuration files, improving the precision of application dependency analysis. In this study, we follow the work of Song et al. [26], modeling not only string manipulation functions but also operations on JavaBeans in JDK, thereby improving the handling of reflection. Additionally, we treat dynamic proxy as a special form of method dispatch and propose a hybrid dispatch approach, thus addressing the gap in existing studies.

6 Conclusion

In this study, we present FLASH, a novel gadget chain detection tool leveraging a deserialization-guided call graph construction approach. In particular, FLASH leverages the results of DDCA to perform hybrid dispatch and reflection analysis, successfully reducing fake edges in the call graph while supplementing missing edges, thereby achieving superior performance in gadget chain detection. Comprehensive experiments demonstrate that FLASH reduces the false negative rate by 30.8% and the false positive rate by 25.9%

(excluding JDD) compared to the state-of-the-art. Furthermore, FLASH detects a total of 90 new gadget chains and 10 existing CVEs. By utilizing newly discovered gadget chains, FLASH is able to expose 5 new exploitation methods of the vulnerabilities, contributing significantly to the domain of Java security.

Acknowledgments

We sincerely thank all anonymous reviewers and our shepherd for their valuable feedback and guidance in improving this paper. This work was sponsored by National Natural Science Foundation of China (No.62372193 and No.U2436207).

Open Science

FLASH, along with all the experimental data, including the newly discovered gadget chains, are all available on our website: https://doi.org/10.5281/zenodo.15606159.

Ethics Considerations

This paper presents a gadget chain detection tool aimed at enhancing the security of Java applications. During development and experiments, we take careful consideration of ethical guidelines and compliance standards to make sure that no ethic concern will be raised in this research. For newly discovered chains, we take responsibility for reporting them to the respective official parties first. We wait for their response and the release of a patch before open-sourcing the details.

References

[1] Rome, 2016. https://mvnrepository.com/artifact/com.rometools/rome.

[2] Hessian, 2022. https://mvnrepository.com/artifact/com.caucho/hessian.

[3] Want to find the implement of fuzz, 2024. https://github.com/fdu-sec/JDD/issues/5.

[4] Darren C Atkinson and William G Griswold. The design of whole-program analysis tools. In *Proceedings of IEEE 18th International Conference on Software Engineering*, pages 16–27. IEEE, 1996.

[5] Darren C Atkinson and William G Griswold. Effective whole-program analysis in the presence of pointers. *ACM SIGSOFT Software Engineering Notes*, 23(6):46–55, 1998.

[6] Eric Bodden. The secret sauce in efficient and precise static analysis: The beauty of distributive, summary-based static analyses (and how to master them). In *Companion Proceedings for the ISSTA/ECOOP 2018 Workshops*, pages 85–93, 2018.

[7] Ahmed Bouajjani, Cezara Drăgoi, Constantin Enea, and Mihaela Sighireanu. On inter-procedural analysis of programs with lists and data. *ACM SIGPLAN Notices*, 46(6):578–589, 2011.

[8] Luca Buccioli, Stefano Cristalli, Edoardo Vignati, Lorenzo Nava, Daniele Badagliacca, Danilo Bruschi, Long Lu, and Andrea Lanzi. Jchainz: Automatic detection of deserialization vulnerabilities for the java language. In *International Workshop on Security and Trust Management*, pages 136–155. Springer, 2022.

[9] Sicong Cao, Biao He, Xiaobing Sun, Yu Ouyang, Chao Zhang, Xiaoxue Wu, Ting Su, Lili Bo, Bin Li, Chuanlei Ma, et al. Oddfuzz: Discovering java deserialization vulnerabilities via structure-aware directed greybox fuzzing. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2726–2743. IEEE, 2023.

[10] Sicong Cao, Xiaobing Sun, Xiaoxue Wu, Lili Bo, Bin Li, Rongxin Wu, Wei Liu, Biao He, Yu Ouyang, and Jiajia Li. Improving java deserialization gadget chain mining via overriding-guided object generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 397–409. IEEE, 2023.

[11] Bofei Chen, Lei Zhang, Xinyou Huang, Yinzhi Cao, Keke Lian, Yuan Zhang, and Min Yang. Efficient detection of java deserialization gadget chains via bottom-up gadget search and dataflow-aided payload construction. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 150–150. IEEE Computer Society, 2024.

[12] Xingchen Chen, Baizhu Wang, Ze Jin, Yun Feng, Xianglong Li, Xincheng Feng, and Qixu Liu. Tabby: Automated gadget chain detection for java deserialization vulnerabilities. In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 179–192. IEEE, 2023.

[13] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In *ECOOP'95—Object-Oriented Programming, 9th European Conference, Åarhus, Denmark, August 7–11, 1995 9*, pages 77–101. Springer, 1995.

[14] George Fourtounis, George Kastrinis, and Yannis Smaragdakis. Static analysis of java dynamic proxies. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 209–220, 2018.

[15] frohoff. ysoserial, 2015. https://github.com/frohoff/ysoserial.

[16] David Grove, Greg DeFouw, Jeffrey Dean, and Craig Chambers. Call graph construction in object-oriented languages. In *Proceedings of the 12th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, pages 108–124, 1997.

[17] Ian Haken. Automated Discovery of Deserialization Gadget Chains, 2018. https://i.blackhat.com/us-18/Thu-August-9/us-18-Haken-Automated-Discovery-of-Deserialization-Gadget-Chains.pdf.

[18] Ben Hardekopf and Calvin Lin. Flow-sensitive pointer analysis for millions of lines of code. In *International Symposium on Code Generation and Optimization (CGO 2011)*, pages 289–298. IEEE, 2011.

[19] Nevin Heintze and Olivier Tardieu. Demand-driven pointer analysis. *ACM SIGPLAN Notices*, 36(5):24–34, 2001.

[20] Pradeep Kumar. JNDI Injection — The Complete Story, 2024. https://infosecwriteups.com/jndi-injection-the-complete-story-4c5bfbb3f6e1.

[21] Zhaojia Lai, Haipeng Qu, and Lingyun Ying. A composite discover method for gadget chains in java deserialization vulnerability. In *QuASoQ/SEED@ APSEC*, pages 10–17, 2022.

[22] Davy Landman, Alexander Serebrenik, and Jurgen J Vinju. Challenges for static analysis of java reflection-literature review and empirical study. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 507–518. IEEE, 2017.

[23] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. Precision-guided context sensitivity for pointer analysis. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–29, 2018.

[24] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. A principled approach to selective context sensitivity for pointer analysis. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 42(2):1–40, 2020.

[25] Yue Li, Tian Tan, Yulei Sui, and Jingling Xue. Self-inferencing reflection resolution for java. In *ECOOP 2014–Object-Oriented Programming: 28th European Conference, Uppsala, Sweden, July 28–August 1, 2014. Proceedings 28*, pages 27–53. Springer, 2014.

[26] Yue Li, Tian Tan, and Jingling Xue. Effective soundness-guided reflection analysis. In *Static Analysis: 22nd International Symposium, SAS 2015, Saint-Malo, France, September 9-11, 2015, Proceedings 22*, pages 162–180. Springer, 2015.

[27] Yue Li, Tian Tan, and Jingling Xue. Understanding and analyzing java reflection. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 28(2):1–50, 2019.

[28] Yifan Luo and Baojiang Cui. Rev gadget: A java deserialization gadget chains discover tool based on reverse semantics and taint analysis. In *International Conference on Emerging Internet, Data & Web Technologies*, pages 229–240. Springer, 2024.

[29] mbechler. Java Unmarshaller Security - Turning your data into code execution, 2017. https://github.com/mbechler/marshalsec.

[30] mbechler. Serianalyzer, 2017. https://github.com/mbechler/serianalyzer.

[31] MITRE. 2024 CWE Top 10 KEV Weaknesses, 2024. https://cwe.mitre.org/top25/archive/2024/2024_kev_list.html.

[32] OpenJDK. Cleanup for handling proxies, 2015. http://hg.openjdk.java.net/jdk8u/jdk8u/jdk/rev/f8a528d0379d.

[33] Oracle. The class File Format, 2013. https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html#jvms-4.1-200-E.1.

[34] Oracle. Dynamic Proxy Classes, 2015. https://docs.oracle.com/javase/8/docs/technotes/guides/reflection/proxy.html.

[35] Shawn Rasheed and Jens Dietrich. A hybrid analysis to detect java serialisation vulnerabilities. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, pages 1209–1213, 2020.

[36] Michael Reif, Florian Kübler, Michael Eichberg, Dominik Helm, and Mira Mezini. Judge: Identifying, understanding, and evaluating sources of unsoundness in call graphs. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 251–261, 2019.

[37] Michael Reif, Florian Kübler, Michael Eichberg, and Mira Mezini. Systematic evaluation of the unsoundness of call graph construction algorithms for java. In *Companion Proceedings for the ISSTA/ECOOP 2018 Workshops*, pages 107–112, 2018.

[38] Thomas Reps. Program analysis via graph reachability. *Information and software technology*, 40(11-12):701–726, 1998.

[39] Thomas Reps, Susan Horwitz, and Mooly Sagiv. Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 49–61, 1995.

[40] Joanna Santos, Mehdi Mirakhorli, and Ali Shokri. Sound call graph construction for java object deserialization. *arXiv preprint arXiv:2311.00943*, 2023.

[41] Joanna CS Santos, Reese A Jones, Chinomso Ashiogwu, and Mehdi Mirakhorli. Serialization-aware call graph construction. In *Proceedings of the 10th ACM SIGPLAN International Workshop on the State of the Art in Program Analysis*, pages 37–42, 2021.

[42] Imen Sayar, Alexandre Bartel, Eric Bodden, and Yves Le Traon. An in-depth study of java deserialization remote-code execution exploits and vulnerabilities. *ACM Transactions on Software Engineering and Methodology*, 32(1):1–45, 2023.

[43] Lei Shang, Xinwei Xie, and Jingling Xue. On-demand dynamic summary-based points-to analysis. In *Proceedings of the Tenth International Symposium on Code Generation and Optimization*, pages 264–274, 2012.

[44] Xiaohu Song, Ying Wang, Xiao Cheng, Guangtai Liang, Qianxiang Wang, and Zhiliang Zhu. Efficiently trimming the fat: Streamlining software dependencies with java reflection and dependency analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–12, 2024.

[45] Johannes Späth, Lisa Nguyen Quang Do, Karim Ali, and Eric Bodden. Boomerang: Demand-driven flow- and context-sensitive pointer analysis for java. In *30th European Conference on Object-Oriented Programming (ECOOP 2016)*. Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2016.

[46] Manu Sridharan and Rastislav Bodík. Refinement-based context-sensitive points-to analysis for java. *ACM SIGPLAN Notices*, 41(6):387–400, 2006.

[47] Manu Sridharan, Denis Gopan, Lexin Shan, and Rastislav Bodík. Demand-driven points-to analysis for java. *ACM SIGPLAN Notices*, 40(10):59–76, 2005.

[48] Prashast Srivastava, Flavio Toffalini, Kostyantyn Vorobyov, François Gauthier, Antonio Bianchi, and Mathias Payer. Crystallizer: A hybrid path analysis framework to aid in uncovering deserialization vulnerabilities. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1586–1597, 2023.

[49] Chenguang Sun and Samuel Midkiff. Demand-driven refinement of points-to analysis. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 264–265. IEEE, 2019.

[50] Tian Tan and Yue Li. Tai-e: A developer-friendly static analysis framework for java by harnessing the good designs of classics. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1093–1105, 2023.

[51] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, and Zheng Wang. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Transactions on Information Forensics and Security*, 16:1943–1958, 2020.

[52] WARATEK. What is a Java Deserialization Vulnerability, 2023. https://waratek.com/blog/java-deserialization-vulnerability.

[53] Junjie Wu, Jingling Zhao, and Junsong Fu. A static method to discover deserialization gadget chains in java programs. In *Proceedings of the 2022 2nd International Conference on Control and Intelligent Robotics*, pages 800–805, 2022.

[54] Dacong Yan, Guoqing Xu, and Atanas Rountev. Demand-driven context-sensitive alias analysis for java. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis*, pages 155–165, 2011.

A Appendix

A.1 Case Study

We use a real gadget chain to explain why FLASH can detect it while the others cannot.

Rome JComponent Chain. Listing 1 presents a new gadget chain in Rome which starts with `JComponent#readObject` method at line 1. It enters the `ArrayTable#put` method after reading the deserialized object. At line 9, the method first checks whether the *key* already exists, specifically by iterating over the existing keys in the table and invoking the `equals` method for comparison, as shown in lines 13-15. An attacker can invoke the `equals` method of class `EqualsBean` in Rome, which subsequently calls `beanEquals` at line 20. Similar to Figure 2 in Section 2.4, it also allows the invocation of any class's '`getter`' methods at line 23. Due to FLASH's reflection analysis, it can automatically discover a new attack vector leading to SSRF, which is the `getContent` method at line 26 since the URL object is controllable by attackers. Therefore, tools that do not perform analysis of reflection may struggle to identify the ultimate exploitation points.

```
1  public Object readObject() { // JComponent
2    int cp = s.readInt();
3    clientProperties = new ArrayTable();
4    for (int c = 0; c < cp; c++) {
5      clientProperties.put(s.readObject(),
       ↪ s.readObject());
6    }
7  }
8  public void put(Object key, Object val) {
9    if (containsKey(key)) ......
10 }
11 public boolean containsKey(Object key) {
12   Object[] arr = (Object[])table;
13   for (int i = 0; i < arr.length-1; i+=2) {
14     if (array[i].equals(key)) ......
15   }
16 }
17 public boolean equals(Object obj) { // EqualsBean
18   return this.beanEquals(obj);
19 }
20 public boolean beanEquals(Object obj) {
21   PropertyDescriptor[] pds =
       ↪ getPropertyDescriptors(this._beanClass);
22   Method pReadMethod = pds[i].getReadMethod();
23   value1 = pReadMethod.invoke(this._obj, NO_PARAMS);
24 }
25 {......}
26 public Object getContent() { // URL
27   return openConnection().getContent(); // SSRF
28 }
```

Listing 1: A new gadget chain within Rome found by FLASH

Additionally, during manual verification of this case, we identify why fuzzing may fail to recognize this chain neither. Listing 2 shows the serialization process of class `JComponent`, which first invokes the `ArrayTable#writeObject` method to write fields. Then at line 8, the `get` method of class `ArrayTable` is called. Similar to line 13-15 in Listing 1, it compares the keys by `equals` method at line 14. As a result, this prematurely triggers the vulnerability during serialization, preventing the effective generation of the attack payload. From this, we can conclude that relying solely on native serialization mechanisms for fuzzing may lead to false negatives. Therefore, directly mutating serialized data could be a promising research direction for reducing false negatives.

A.2 Other Benefits of FLASH

This subsection mainly analyzes other benefits that our approach can bring to the security researchers.

Call Graph Soundness. Call graphs are essential in program analysis, serving as the foundation for tasks such as interprocedural analysis, vulnerability detection, and program optimization [7, 16, 51]. The soundness of a call graph indicates that *all possible program behaviors are modeled* [36, 37], resulting in fewer missing edges. Specifically, we evaluate the soundness of the deserialization call graph constructed by FLASH, following the methodology of Santos et al. [40].

```
1  private void writeObject(ObjectOutputStream s) { //
   ↪ ArrayTable
2    s.defaultWriteObject();
3    ArrayTable.writeArrayTable(s, clientProperties);
4  }
5  static void writeArrayTable(ObjectOutputStream s,
   ↪ ArrayTable table) {
6    for (Object key : table.getKeys(null)) {
7      s.writeObject(key);
8      s.writeObject(table.get(key));
9    }
10 }
11 public Object get(Object key) {
12   Object[] arr = (Object[])table;
13   for (int i = 0; i < arr.length-1; i+=2) {
14     if (array[i].equals(key)) ......
15   }
16 }
```

Listing 2: Serialization of JComponent class

In particular, we use the Java Call-graph Assessment & Test Suite (CATS), which is the dataset widely used to evaluate call graph soundness [36, 37]. CAST is derived from an extensive analysis of real Java projects and contains 12 test cases for assessing serialization/deserialization call graphs where each test case annotates the expected methods for a given call site. We selected all the 7 cases related to deserialization, and Table 5 shows the results. FLASH successfully passes all tests, demonstrating the soundness of its deserialization call graph. We plan to construct a more comprehensive dataset in future to further validate the soundness of our approach. Therefore, security researchers can not only leverage the deserialization call graphs constructed by FLASH for further studies on deserialization scenarios (such as fuzzing payload validation, call graph optimization, and so on), but also can extend our DDCA approach, along with its support for dynamic proxy and reflection, to the detection of other similar vulnerability scenarios (e.g., injection vulnerability in web applications).

Patch Soundness. The gadget chains detected by FLASH provide valuable insights for security researchers in comprehensive patching of JDVs. As shown in Table 1, FLASH identifies 90 new gadget chains, indicating the presence of more hidden gadget chains in existing applications. The newly discovered gadget chains may bypass both blacklist-based mitigation strategies and direct code-level patches. For example, in Section 4.5, CVE-2023-23268 utilizes a new gadget chain to bypass the blacklist-based strategy but FLASH identified a new gadget chain that can result in a SSRF attack across all JDK versions. Therefore, FLASH can provide significant value to both developers and security defenders.

Table 5: Results of test cases applying to FLASH

Approach	Ser4	Ser5	Ser7	Ser8	Ser9	ExtSer2	ExtSer3
FLASH	✓	✓	✓	✓	✓	✓	✓